

What Do Cancer Driver Mutations Break Inside a Protein Language Model?

Using ESMC's sparse-autoencoder features to mechanistically interpret — and predict — the functional impact of cancer mutations.

Entry point for the project; written for both a human and Claude Code. If you are Claude Code: read the whole file, then start at §11 **First Session**. Do **not** begin coding before resolving the hardware path in §9 — it determines the entire data-generation strategy.

1. Thesis (one paragraph)

Protein language models predict the functional impact of mutations well, but as black boxes — a likelihood score with no account of *what* the mutation disrupts. The 2026 release of ESMC's sparse autoencoder (SAE) changed the substrate: it decomposes the model's internal representation into ~16,000 individually interpretable, GPT-5-annotated biological features. Someone has already shown these features work as a "semantic signature" for enzyme function. **Nobody has yet asked what happens to those features when a cancer driver mutation hits a protein.** This project extracts SAE-feature activations for wild-type vs. mutant cancer proteins, quantifies which interpretable features each mutation disrupts, tests whether that disruption distinguishes drivers from passengers (and pathogenic from benign) — and, critically, whether the *interpretable* feature view explains or improves on a raw likelihood baseline. The interpretability is the contribution; a clean negative on prediction is still publishable.

2. Why this is worth doing (the gap)

- SAEs on protein LMs are established (InterPLM 2024; PNAS 2025 mapped ESM-2 SAE features to Gene Ontology terms), so the method is validated.
- The ESMC SAE is far more capable than prior PLM SAEs (built on a 6B model, 16,384 features, agent-annotated) and is only months old.
- A downstream-application template exists and works: ESMC-SAE features → enzyme EC prediction without task-specific training (Hu et al., arXiv 2606.12209, June 2026).
- **The cancer-mutation application is unworked.** Existing cancer-SAE work (xAEDriver) operates on DepMap dependency/expression data, not on protein-LM sequence representations. This is open ground with a proven method and an active adjacent literature to cite.
- It fits the hardware constraints: feature extraction over a bounded mutant set is an overnight GPU batch job, not a training run.

3. Research question & hypotheses

RQ: Do ESMC SAE features encode the functional properties that cancer driver mutations disrupt, and does feature-level disruption explain or improve discrimination of driver-vs-passenger / pathogenic-vs-benign beyond the model's raw likelihood?

- **H1** — Driver/pathogenic mutations cause larger and more *concentrated* disruption of specific SAE features than passenger/benign mutations (measured as change in feature activation between WT and mutant).
- **H2** — The disrupted features are biologically coherent per their GPT-5 descriptions (catalytic, binding-site, structural) and differ systematically between oncogenes (often gain-of-function, recurrent hotspots) and tumor suppressors (often loss-of-function, dispersed). This is the interpretability payoff.
- **H3** — A classifier on SAE-feature-disruption vectors matches or beats raw masked-marginal likelihood for driver/pathogenicity calling. **If it does not**, the SAE features still provide a mechanistic explanation likelihood alone cannot — report that explicitly rather than burying it.

4. Definition of done

1. A reproducible pipeline: curated cancer variant set → WT/mutant SAE feature activations → disruption features → classifier + interpretation, regenerable by one command.
2. Per-mutation SAE-feature-disruption profiles for a curated cancer-gene panel.
3. Discrimination scores (driver-vs-passenger and pathogenic-vs-benign) for: raw likelihood baseline, SAE-disruption classifier, and a combined model — each on leakage-proof splits.
4. An interpretability analysis: which features drive correct calls (SHAP over the SAE-disruption features), and whether oncogene vs. tumor-suppressor mechanisms separate in feature space (H2).
5. Write-up → bioRxiv / ML-for-bio workshop.

Non-goals: training ESMC; structure prediction or binder design; any wet-lab or clinical claim; whole-proteome scale (curated panel only).

5. Methods

5.1 Core pipeline

1. **Curate variants.** A panel of cancer genes (start ~50–100: classic oncogenes + tumor suppressors), each with labeled variants (driver/passenger, pathogenic/benign, oncogene/TSG mechanism).

2. **Generate sequences.** Wild-type protein + one sequence per missense variant.
3. **Extract SAE features.** Run ESMC, pull the SAE layer's activations (per §9), decode to the 16,384-feature codebook for WT and each mutant. Residue-level (Top-K=64 per residue) and pooled per-protein.
4. **Compute disruption.** Per-variant feature-disruption vector = change in feature activation WT→mutant (at and around the mutated residue, and pooled). This is the core engineered representation.
5. **Classify + interpret.** XGBoost / Extra Trees on disruption vectors → driver/pathogenicity. SHAP to identify which interpretable features drive calls; cross-reference the GPT-5 feature descriptions.

5.2 Baselines (the spine — keep this honest)

- **Raw likelihood baseline:** ESMC/ESM-2 masked-marginal log-likelihood ratio (WT vs mutant) — the standard zero-shot VEP score. Everything is reported relative to this. If SAE features don't beat it on prediction, say so and pivot the claim to interpretation.
- **Embedding baseline:** plain ESMC embedding deltas (no SAE) into the same classifier — isolates what the *sparse/interpretable* decomposition adds over dense embeddings.

5.3 Metrics

- AUROC / AUPRC for driver-vs-passenger and pathogenic-vs-benign.
- Calibration check against ProteinGym (so the likelihood baseline is known-good before comparison).
- Feature-disruption concentration (sparsity/entropy of the disruption vector) as an H1 quantity.
- Interpretability: agreement between top-SHAP features' GPT-5 descriptions and known mechanism (manual + systematic).

6. Evaluation protocol

- **Held-out genes:** train on some cancer genes, test on unseen ones (the real generalization test — avoids memorizing per-gene quirks).
- **Held-out variants within seen genes:** secondary.
- Stratify by mechanism (oncogene/TSG), by variant recurrence (hotspot vs. rare), and by protein-family representation in ESMC training. Fixed seeds; splits committed; no test-set tuning. Watch label leakage: many "driver" labels derive from recurrence, which correlates with the features — document this.

7. Data sources (all public)

Source	Use
COSMIC / Cancer Hotspots (cancerhotspots.org)	Recurrent somatic driver mutations
OncoKB	Oncogenicity + oncogene/TSG mechanism labels
ClinVar	Pathogenic/benign germline missense labels
ProteinGym	DMS benchmark to calibrate the likelihood baseline
UniProt	WT canonical sequences; functional-site annotations for H2 cross-check
ESMC SAE codebook + GPT-5 feature descriptions (HuggingFace, Biohub(esm))	The interpretable feature definitions

8. Repository structure

```
esmc-sae-cancer/
├─ PROJECT_BRIEF_ESMC_SAE.md    # this file
├─ CLAUDE.md                    # conventions (see §10)
├─ pyproject.toml
├─ data/
│   └─ raw/                     # COSMIC/ClinVar/OncoKB/UniProt dumps
(gitignored)
│   └─ variants/                # curated, labeled variant table + committed
splits
│   └─ features/                # cached SAE activations (large; gitignored)
├─ src/sae_cancer/
│   └─ variants/                # curation, labeling, sequence generation
│   └─ esmc/                   # model loading, activation extraction, SAE
decode
│   └─ disruption/              # WT→mutant feature-disruption computation
│   └─ baselines/               # masked-marginal likelihood, embedding delta
│   └─ models/                  # XGBoost / Extra Trees
│   └─ interpret/               # SHAP + feature-description cross-reference
│   └─ eval/                    # splits, metrics, calibration
```

```
|— experiments/  
|— results/
```

```
# one yaml per run
```

9. Hardware path — RESOLVE THIS FIRST (it drives everything)

Hardware: desktop, RTX 4070 (~12 GB) + RTX 4060 (~8 GB).

The flagship released SAE is `ESMC-6B-sae-layer60-k64-codebook16384` — it needs **ESMC 6B layer-60 activations**. Because you need activations for *mutant* sequences (absent from any precomputed atlas), you cannot avoid running the model. ESMC 6B at FP16 is ~12 GB of weights alone — tight on the 4070 with no headroom. **Claude Code, evaluate these three paths in Phase 0 and report before proceeding:**

- **Path A — 6B inference on a bounded set (preferred if it runs).** Load ESMC 6B with 8-bit quantization and/or CPU offload; run *inference only* (no optimizer state) over a bounded mutant set (~50-100 genes × their labeled variants = $O(10^4)$ sequences). Slow but overnight-feasible. Keeps the agent-annotated codebook. Benchmark throughput and peak VRAM on a 50-sequence smoke test first.
- **Path B — smaller released SAE.** Biohub released SAEs at multiple model scales/layers; check HuggingFace for one built on a model size the 4070 runs comfortably (e.g. an ESMC-600M-based SAE). If it exists, this is the path of least resistance. Confirms availability before committing.
- **Path C — train your own SAE on ESMC-600M.** ESMC 600M runs easily on the 4070; SAE training is cheap relative to the base model and is an overnight job. Full control and an "accessible/reproducible on consumer hardware" angle, but you lose the GPT-5 feature descriptions and must interpret features yourself.

Decision rule: try B, fall back to A, use C if A is too slow or B unavailable. Document which path and why.

Run first, report back:

```
bash  
  
nvidia-smi  
python -c "import torch; print(torch.__version__, torch.cuda.is_available()), to  
# then: check HF for ESMC SAE variants by model scale
```

Recommended: WSL2 + CUDA on the Windows box for this stack. Core deps: Python 3.11, PyTorch (CUDA), the `esm` package (Biohub/ESMC), `bitsandbytes` (quantization), `xgboost`, `scikit-learn`, `shap`, `pandas`, `biopython`. Prefer `uv`.

10. Engineering conventions (also in CLAUDE.md)

- Reproducibility is the product: one command regenerates every number; fixed seeds; committed splits.
- Score everything against the raw-likelihood baseline. SAE features must *earn* their complexity.
- Cache SAE activations aggressively — they are the expensive artifact; never recompute silently.
- Held-out-**genes** is the headline split. Beware recurrence-driven label leakage.
- Notebooks are scratch; anything that matters becomes tested code in `(src/)`.
- Start on a 5-gene smoke-test panel; scale the panel only once the full pipeline is green.
- Interpretation claims (H2) must cite the actual feature description and a UniProt functional-site cross-check — no hand-waving "this feature looks like a binding site."

11. First session (in order)

Phase 0 — Hardware path + scaffold. Resolve §9 (B→A→C) and report. Scaffold repo, `(pyproject.toml)`, `(CLAUDE.md)`, `(.gitignore)`. Verify CUDA. Smoke-test SAE feature extraction on a single WT protein and confirm you get a 16,384-dim (or chosen-codebook) activation vector.

Phase 1 — Variants + baseline. Curate a 5-gene smoke panel (e.g. KRAS, TP53, BRAF, PTEN, EGFR) with labeled variants from OncoKB/ClinVar/Cancer Hotspots. Implement the masked-marginal likelihood baseline and the ProteinGym calibration check. First results table: likelihood baseline on held-out variants.

Phase 2 — SAE disruption features. Generate WT/mutant sequences, extract SAE activations, compute disruption vectors. Sanity-check H1 on the smoke panel (do known drivers disrupt features more?).

Phase 3 — Classify + interpret. XGBoost on disruption vectors vs. both baselines (likelihood, embedding-delta). SHAP. Cross-reference top features against GPT-5 descriptions + UniProt sites.

Phase 4 — Scale + mechanism. Expand to the full ~50-100-gene panel. Test H2 (oncogene vs TSG separation). Held-out-genes evaluation. Draft results.

Stop and surface findings after each phase.

12. Known pitfalls

- **You must run the model on mutants** — there is no precomputed shortcut for novel mutant sequences. The hardware path is the make-or-break decision, not an afterthought.

- **Label leakage via recurrence** — "driver" labels often derive from mutational recurrence, which the model may already encode. Held-out-genes splits and stratification by recurrence are the guard.
- **Interpretation is seductive and easy to overclaim** — a feature firing near a mutation is not proof of mechanism. Require description + functional-site cross-check.
- **SAE features can be redundant / split** (feature absorption) — don't treat the 16,384 dims as cleanly orthogonal concepts.
- **Activation caches are large** — budget disk; gitignore them; key them by (model, layer, sequence-hash).

13. Selected references

- Candido et al. (2026): ESMC-6B and its companion SAE ([ESMC-6B-sae-layer60-k64-codebook16384](#)); Biohub/EvolutionaryScale ([esm](#)) release.
- Hu et al. (2026): Interpretable enzyme function prediction via ESMC SAE features, arXiv 2606.12209 — the methodological template.
- Simon & Zou (2024): InterPLM — SAEs on ESM-2.
- PNAS (2025): SAE features in PLM representations map to Gene Ontology terms.
- Brandes et al.: genome-wide variant effect prediction with ESM1b (likelihood-baseline lineage).
- ProteinGym (Notin et al.): standard DMS/VEP benchmark.

Scope note: computational interpretability research. SAE-feature attributions and driver predictions are hypotheses, not clinical or validated findings.